Trabajo 2: Redes neuronales para datos tabulares (¿me prestas?)

Profesor: **Juan David Ospina Arango**

Monitor: Andrés Mauricio Zapata Rincón

Universidad Nacional de Colombia

Facultad de Minas

Departamento de Ciencias de la Computación y de la Decisión

Semestre 2026-02

Curso: Redes neuronales y algoritmos bioinspirados

Modalidad: trabajo en equipo, con sustentación oral y evaluación individual

---

Propósito

Un banco quiere saber dos cosas antes de prestarle dinero a alguien: qué tan probable es que no le paguen y, si no le pagan, cuánto va a perder. En este trabajo van a responder ambas preguntas con redes neuronales artificiales: la primera es un problema de **clasificación binaria** y la segunda un problema de **regresión**. Además, van a convertir su modelo en una herramienta que cualquier persona pueda usar y van a salir a venderla.

Las redes neuronales no siempre son la mejor herramienta para datos tabulares. Parte del reto es averiguar, con evidencia y comparaciones justas, qué aportan en este problema y qué no.

Pueden (y se espera) que usen herramientas de inteligencia artificial. Pero la IA se equivoca, a veces con total seguridad. Parte del trabajo consiste en demostrar que ustedes entienden lo suficiente como para darse cuenta. Lo que se evalúa no es solo lo que entregan, sino lo que son capaces de explicar, modificar y defender en vivo.

Datos

Usarán el conjunto *Credit Risk Analysis* de Kaggle, con créditos de consumo otorgados por la plataforma Lending Club: https://www.kaggle.com/datasets/ranadeep/credit-risk-dataset

Reporten en el texto el número de registros y variables con que trabajaron, el periodo que cubren y la fecha de descarga. Consulten el diccionario de variables incluido en el conjunto y citen su significado desde esa fuente, no desde lo que una IA les diga que significan.

Parte 1: Probabilidad de incumplimiento (clasificación binaria)

Variable objetivo

Modelarán la probabilidad de que un cliente sea "malo", es decir, que incumpla con el pago de su crédito. Construyan una variable binaria a partir de `loan_status` con la siguiente codificación:

Categoría original	Codificación	Justificación
Fully Paid	0	Buen pagador confirmado.
Does not meet the credit policy. Status:Fully Paid	0	No cumple requisitos, pero pagó: buen pagador confirmado.
Charged Off	1	Crédito reconocido como incobrable.
Default	1	Impago confirmado.
Late (31-120 days)	1	Se considera mal pagador.
Does not meet the credit policy. Status:Charged Off	1	No cumple requisitos y no pagó.
Current	NA	Está al día; no hay certeza sobre su comportamiento final.
Issued	NA	Aprobado, pero sin historia de pago.
In Grace Period	NA	Aún no está obligado a pagar.
Late (16-30 days)	NA	No hay certeza sobre su comportamiento final.

Los casos NA no se incluyen en el entrenamiento. Los modelos de riesgo de crédito estiman la probabilidad de ser malo (valor 1), no la de ser bueno.

Tareas

1. **Caso de uso.** Describan quién usa el modelo, para qué decisión, en qué momento y con qué información disponible en ese momento. Todas las decisiones de modelamiento posteriores deben ser coherentes con este caso de uso.
2. **Análisis descriptivo.** Exploren los datos y formulen hipótesis explícitas sobre qué hace más riesgosa a una persona. Al final del trabajo, digan cuáles se confirmaron y cuáles no.
3. **Modelo de referencia.** Ajusten una **regresión logística** como modelo de baja complejidad.
4. **Red neuronal.** Construyan un perceptrón multicapa para estimar la probabilidad de incumplimiento y **optimicen su arquitectura e hiperparámetros**: número de capas, neuronas por capa, funciones de activación, regularización (por ejemplo, *dropout*, penalización L_2, parada temprana), optimizador y tasa de aprendizaje. El método de búsqueda es libre, pero debe estar justificado.
5. **Evaluación.** Definan y justifiquen sus conjuntos de entrenamiento, validación y prueba, incluyendo al menos una **partición temporal** según la fecha de originación. Evalúen:
    - **discriminación:** AUC, índice de Gini y estadístico KS;
    - **calibración:** puntaje de Brier y curva de calibración.
6. **Scorecard.** Representen el modelo con una scorecard. Definan el puntaje base, las odds de referencia y los puntos para duplicar las odds (PDO), y deriven la transformación de probabilidad a puntaje. Expliquen cómo obtienen los puntos a partir de su modelo. Las scorecards pueden construirse con variables continuas.
7. **Variables de riesgo.** Analicen qué variables hacen más riesgosa a una persona, con un método de interpretabilidad justificado y citado.

Parte 2: Pérdida dado incumplimiento (regresión)

Cuando un cliente incumple, el banco suele recuperar parte del dinero. La **pérdida dado incumplimiento** (LGD, por sus siglas en inglés) es la fracción de la exposición que finalmente se pierde.

1. **Variable objetivo.** Para los créditos que incumplieron, construyan la LGD a partir de las variables del conjunto. Escriban la fórmula, justifiquen cada campo que usan y discutan sus limitaciones.
2. **Modelo de referencia.** Ajusten un modelo de baja complejidad, por ejemplo una regresión lineal.
3. **Red neuronal.** Construyan y optimicen un perceptrón multicapa para estimar la LGD. Justifiquen la elección de la función de pérdida y de la activación de la capa de salida a partir de la naturaleza de la variable objetivo.
4. **Evaluación.** Reporten MAE y RMSE, y comparen la distribución de las predicciones con la distribución observada.
5. **Pérdida esperada.** Combinen ambos modelos para estimar la pérdida esperada de cada crédito,

$$E[L] = PD \cdot LGD \cdot EAD,$$

6. definiendo y justificando la exposición al incumplimiento (EAD) que usan.

Regla de comparación justa

Para comparar modelos deben hacerlo bajo condiciones explícitas y equivalentes:

- **Mismos datos:** todos los modelos de una misma tarea usan el mismo preprocesamiento y las mismas particiones.
- **Presupuesto de búsqueda explícito:** reporten cuántas configuraciones entrenaron para cada familia de modelos y el tiempo de cómputo total. Una arquitectura que "gana" tras 200 intentos contra un modelo de referencia ajustado una sola vez no es una comparación justa sin esa aclaración.
- **Varias corridas independientes:** entrenen la configuración final de cada red con **al menos 30 semillas distintas** y reporten media, desviación estándar, mejor y peor valor de cada métrica.
- **Complejidad:** reporten el número de parámetros entrenables de cada modelo.

Discusión

¿Qué aportó la red neuronal frente al modelo de referencia en cada tarea? Respondan con base en las métricas, su variabilidad entre semillas, el costo de cómputo y la interpretabilidad. Incluyan un listado de **aprendizajes sobre el problema** que surgieron del proceso de modelamiento.

Entregables

1. Reporte técnico publicado como entrada de blog

El reporte debe contener el entendimiento desarrollado en el trabajo, la metodología seguida y justificada, los resultados y la discusión. Se exige:

- **Bibliografía en formato APA (7.ª edición).** Todas las referencias deben existir, ser verificables y estar citadas en el texto. Una referencia inexistente se penaliza.
- **Figuras y tablas numeradas, con título y fuente**, y todas citadas explícitamente en el texto (por ejemplo, "como se observa en la Figura 3..."). No se aceptan figuras huérfanas.
- **Fundamentación matemática:**
    - definición de neurona artificial, perceptrón multicapa y cada función de activación utilizada;
    - derivación de cada función de pérdida a partir de un modelo probabilístico (máxima verosimilitud);

    - el algoritmo de retropropagación y las ecuaciones de actualización del optimizador utilizado;
    - la formulación de cada técnica de regularización;
    - la definición de cada métrica de evaluación;
    - la derivación de la scorecard.
- **Sección de uso de IA:** los principales prompts utilizados y una discusión de su impacto en el resultado final.
- **Sección "Cacería de la alucinación"** (ver abajo).
- **Enlaces** al repositorio Git, a la aplicación web y al video promocional.

2. Repositorio Git reproducible

El repositorio debe permitir que cualquier persona reproduzca los resultados. Requisitos mínimos:

- `README` con instrucciones de instalación y ejecución.
- Entorno especificado con versiones fijas (`requirements.txt`, `environment.yml` o similar).
- **Semillas aleatorias fijas** y documentadas.
- Los datos procesados que usan los modelos **guardados en el repositorio**, junto con el código que los genera desde el archivo original, para que la ejecución no dependa de conexión a internet ni de APIs externas. Si el archivo original excede los límites de tamaño del repositorio, guarden una versión procesada y comprimida, y documenten cómo se obtuvo.
- Los modelos entrenados guardados, de modo que la aplicación y las figuras no requieran reentrenar.
- Las figuras deben poder regenerarse desde el código.
- Historial de commits que refleje el trabajo real del equipo. Un único commit al final no es aceptable.

3. Todo debe correr en vivo

Durante la sustentación el equipo ejecutará su código frente al profesor. Por eso:

- Cada experimento principal debe lanzarse con **un solo comando**.
- Los parámetros (arquitectura, tasa de aprendizaje, regularización, número de épocas, PDO, puntaje base, etc.) deben poder cambiarse desde la línea de comandos o un archivo de configuración, sin editar el código fuente.
- Una corrida de demostración debe tardar **menos de dos minutos**. Si el entrenamiento completo tarda más, preparen un modo de demostración (por ejemplo, con una muestra de los datos) que se lance también con un solo comando.
- La aplicación web debe poder levantarse localmente con un solo comando.
- Antes de la sustentación, prueben todo en un **clon limpio** del repositorio en una máquina distinta a la de desarrollo.

4. Aplicación web

Una aplicación publicada en la que una persona ingrese sus características y obtenga:

- su puntaje en la scorecard y su probabilidad de incumplimiento;
- cómo se ubica frente a la población (por ejemplo, su percentil sobre la distribución de puntajes);
- cómo cambiaría su puntaje si cambiara alguna de sus características.

La aplicación debe ser intuitiva para un usuario nuevo y contener los enlaces al reporte técnico y al video promocional.

5. Video promocional

Un video de **2 a 3 minutos** que presente la aplicación como la solución a un problema y genere entusiasmo por usarla. **Todos los integrantes del equipo deben aparecer en el video promocionando el trabajo.** Saber vender lo que uno construye también se aprende.

6. Videos de contribución individual

Cada integrante graba un video de **2 a 3 minutos** hablando en primera persona sobre sus aportes específicos. El video debe **mostrar evidencia concreta**: un fragmento de código que escribió, un commit, una figura que elaboró o una sección del reporte que redactó, y explicar brevemente cómo funciona.

Lo que cada persona declare en su video podrá ser preguntado en la sustentación.

7. Cacería de la alucinación 🎯

Las herramientas de IA producen errores que suenan convincentes. Su misión es cazarlos.

Documenten **al menos cinco hallazgos** en los que una IA generó algo incorrecto, impreciso, desactualizado o subóptimo durante el desarrollo del trabajo. Deben cubrir al menos tres de estas categorías:

- **Matemáticas:** derivadas, funciones de pérdida, fórmulas de métricas o de la scorecard.
- **Código:** errores de implementación, preprocesamiento defectuoso, métricas mal calculadas.
- **Datos del mundo real:** significado de las variables, definiciones del negocio de crédito, normativa.
- **Bibliografía:** referencias inexistentes, autores o años equivocados, DOI que no resuelven.
- **Conceptos:** afirmaciones falsas sobre las redes neuronales, su entrenamiento o su desempeño en datos tabulares.

Para cada hallazgo, reporten:

1. El prompt utilizado (literal).
2. La parte relevante de la respuesta de la IA.
3. Cómo sospecharon que había un error.
4. La evidencia que lo demuestra: una derivación, un experimento, una fuente primaria.
5. La corrección.

6. La lección aprendida: ¿qué verificación habría evitado el error?

Si no logran encontrar cinco hallazgos genuinos, documenten las verificaciones que hicieron y sus resultados. Un hallazgo inventado o sin evidencia se penaliza más que un hallazgo faltante.

Al final del curso se premiará **la mejor alucinación cazada**: la más sutil, la mejor documentada o la más peligrosa si hubiera pasado desapercibida.

Sustentación

Cada equipo tiene **10 minutos**:

Momento	Duración	Contenido
Demostración	2 min	Ejecución en vivo de un experimento, con un solo comando.
Modificación en vivo	3 min	El profesor pide un cambio de parámetro o de configuración. **Antes de ejecutarlo**, el equipo predice qué ocurrirá y por qué.
Preguntas	5 min	El profesor pregunta a **cualquier integrante**, escogido al azar, sobre cualquier parte del trabajo, incluyendo lo declarado en los videos individuales.

Todos los integrantes deben estar en capacidad de explicar todo el trabajo, no solo su parte.

Evaluación

Componente	Peso	Tipo
Reporte técnico (rigor, APA, figuras y tablas, discusión)	20 %	Grupal
Repositorio y reproducibilidad	5 %	Grupal
Aplicación web	10 %	Grupal
Video promocional	5 %	Grupal
Ejecución y modificación en vivo	10 %	Grupal
Cacería de la alucinación	15 %	Grupal

Video de contribución individual	5 %	Individual
Respuestas en la sustentación	30 %	Individual

La nota individual de la sustentación puede ajustar la nota grupal si un integrante no logra explicar un componente que declaró como propio.